Projet JavaScript – Gestion d'une boutique en ligne

Contexte général

Tu vas construire progressivement une petite application de gestion d'une boutique en ligne. Ce projet te permettra de mettre en pratique toutes les notions du cours : types, variables, opérateurs, conditions, boucles, fonctions, tableaux, objets, et asynchrone.

Le projet se divise en plusieurs étapes, chacune correspondant à une ou plusieurs parties du cours.

Étape 1 – Installation et premiers pas avec Node.js

Objectif : Installer Node et créer le fichier principal du projet.

Consignes

- Installe Node.js si ce n'est pas déjà fait.
- Crée un fichier `shop.js` qui affiche :
  - Un message de bienvenue : `"Bienvenue dans la boutique en ligne !"`
  - La version de Node avec `process.version`
- Exécute le fichier avec `node shop.js` pour vérifier que tout fonctionne.

Étape 2 – Types primitifs et variables

Objectif : Déclarer les premières variables du projet avec différents types.

Consignes

Dans `shop.js`, déclare les variables suivantes :

```
const shopName = "TechStore";           // string
let itemCount = 0;                      // number
let isOpen = true;                      // boolean
let specialOffer = null;                // null
let nextDelivery;                       // undefined
const maxStock = 1000n;                 // bigint
```

Affiche chaque variable avec `console.log` en précisant son type en commentaire.

Étape 3 – Objets et tableaux

Objectif : Créer la structure de données pour les produits et le panier.

Consignes

Dans `shop.js`, crée :

1. **Un objet produit :**

```
const product1 = {
  id: 1,
  name: "Clavier mécanique",
  price: 79.99,
  stock: 15,
  category: "Périphériques"
};
```

2. **Un tableau de produits :**

```
const products = [
  { id: 1, name: "Clavier mécanique", price: 79.99, stock: 15 },
  { id: 2, name: "Souris gaming", price: 49.99, stock: 23 },
  { id: 3, name: "Écran 24 pouces", price: 199.99, stock: 8 },
  { id: 4, name: "Casque audio", price: 89.99, stock: 12 }
];
```

3. **Un panier vide :**

```
let cart = [];
```

Affiche le nombre de produits disponibles et le premier produit du catalogue.

Étape 4 – Détection de types avec `typeof`

Objectif : Vérifier les types des données manipulées.

Consignes

Crée une fonction `checkType(value)` qui :

- Affiche la valeur
- Affiche son type avec `typeof`
- Si c'est `null`, affiche `"null"` (car `typeof null` renvoie `"object"`)
- Si c'est un tableau, affiche `"array"` (avec `Array.isArray()`)

Teste la fonction avec plusieurs valeurs : un nombre, une chaîne, un objet, un tableau, `null`, `undefined`.

Étape 5 – Nommage et portée des variables

Objectif : Respecter les bonnes pratiques de nommage et comprendre la portée.

Consignes

1. Renomme les variables si nécessaire pour suivre la convention `camelCase`.
2. Crée une fonction `displayShopInfo()` qui :
   - Déclare une variable locale `let message = "La boutique est ouverte";`
   - Affiche `message` et `shopName` (variable globale)
3. Appelle la fonction et vérifie que `message` n'est pas accessible en dehors.

Bonus : Crée un bloc `if` qui déclare une variable `let discount = 10;` à l'intérieur, et observe qu'elle n'est pas accessible en dehors (shadowing).

Étape 6 – Opérateurs arithmétiques et d'affectation

Objectif : Calculer des prix, des totaux, des réductions.

Consignes

Crée une fonction `calculateTotal(price, quantity, discount = 0)` qui :

- Calcule le sous-total : `price * quantity`
- Applique une réduction en pourcentage si fournie
- Retourne le total final

Exemples d'utilisation :

```
console.log(calculateTotal(79.99, 2));        // 159.98
console.log(calculateTotal(79.99, 2, 10));    // 143.98 (avec 10% de réduction)
```

Bonus : Utilise des opérateurs composés (`+=`, `*=`) pour gérer un compteur de commandes.

Étape 7 – Opérateurs de comparaison et logiques

Objectif : Vérifier la disponibilité des produits et les conditions d'achat.

Consignes

Crée une fonction `canAddToCart(product, quantity)` qui retourne `true` si :

- Le produit est en stock (`product.stock > 0`)
- La quantité demandée est disponible (`quantity <= product.stock`)
- La quantité est supérieure à 0

Utilise des opérateurs logiques (`&&`, `||`) et de comparaison (`===`, `>=`, etc.).

Teste la fonction avec différents cas :

```
console.log(canAddToCart(products[0], 5));   // true
console.log(canAddToCart(products[0], 20));  // false
console.log(canAddToCart(products[0], 0));   // false
```

Étape 8 – Conditions : `if`, `else`, ternaire

Objectif : Gérer différents scénarios d'achat.

Consignes

Crée une fonction `getShippingCost(total)` qui :

- Retourne `0` si le total est supérieur ou égal à 50€ (livraison gratuite)
- Retourne `5.99` si le total est entre 20€ et 50€
- Retourne `8.99` si le total est inférieur à 20€

Utilise `if / else if / else` ou l'opérateur ternaire selon ce qui te semble le plus lisible.

Teste avec plusieurs montants :

```
console.log(getShippingCost(60));   // 0
console.log(getShippingCost(35));   // 5.99
console.log(getShippingCost(15));   // 8.99
```

Étape 9 – Switch : catégories de produits

Objectif : Attribuer une icône ou une couleur selon la catégorie.

Consignes

Crée une fonction `getCategoryIcon(category)` qui utilise `switch` pour retourner une icône (emoji ou symbole) selon la catégorie :

- `"Périphériques"` → `" 🖱 "`
- `"Audio"` → `" 🎧 "`
- `"Écrans"` → `" 🖥 "`
- `"Accessoires"` → `" 🎒 "`
- Par défaut → `" 📦 "`

Teste la fonction avec différentes catégories.

Étape 10 – Boucle `for` classique

Objectif : Parcourir le catalogue de produits.

Consignes

Crée une fonction `displayCatalog()` qui :

- Parcourt le tableau `products` avec une boucle `for` classique
- Affiche pour chaque produit : `"[ID] Nom - Prix€ (Stock: X)"`

Exemple d'affichage :

```
[1] Clavier mécanique - 79.99€ (Stock: 15)
[2] Souris gaming - 49.99€ (Stock: 23)
...
```

Étape 11 – `forEach` : affichage simplifié

Objectif : Utiliser `forEach` pour parcourir le catalogue.

Consignes

Réécris la fonction `displayCatalog()` en utilisant `forEach` à la place du `for` classique.

Compare les deux versions : laquelle préfères-tu et pourquoi ? (Note ta réponse en commentaire.)

Étape 12 – Fonctions classiques et fléchées

Objectif : Créer des fonctions utilitaires pour la boutique.

Consignes

Crée les fonctions suivantes (tu peux alterner fonctions classiques et fléchées) :

1. `addToCart(product, quantity)` : ajoute un produit au panier (tableau `cart`) sous forme d'objet `{ product, quantity }`
2. `removeFromCart(productId)` : retire un produit du panier par son ID
3. `getCartTotal()` : calcule le total du panier (somme de `product.price * quantity` pour chaque élément)

Utilise des fonctions fléchées pour les fonctions courtes (par exemple pour `getCartTotal` avec `.reduce()`).

Teste les fonctions :

```javascript
addToCart(products[0], 2);
addToCart(products[1], 1);
console.log("Total panier :", getCartTotal());
removeFromCart(1);
console.log("Total après suppression :", getCartTotal());
```

Étape 13 – Méthodes de tableaux

Objectif : Manipuler le catalogue et le panier avec les méthodes de tableaux.

Consignes

Crée les fonctions suivantes :

1. `findProductById(id)` : retourne le produit correspondant à l'ID (utilise `.find()`)
2. `getProductsByCategory(category)` : retourne tous les produits d'une catégorie (utilise `.filter()`)
3. `getProductNames()` : retourne un tableau avec uniquement les noms des produits (utilise `.map()`)
4. `isProductInStock(id)` : vérifie si un produit est en stock (utilise `.some()`)
5. `getTotalStock()` : retourne le stock total de tous les produits (utilise `.reduce()`)

Teste chaque fonction avec des exemples concrets.

Étape 14 – Objets littéraux : profil utilisateur

Objectif : Gérer les informations client.

Consignes

Crée un objet `customer` avec :

- `firstName`, `lastName`, `email`, `isVIP` (booléen)
- Un tableau `orders` (historique des commandes)
- Une méthode `getFullName()` qui retourne le nom complet

Ajoute dynamiquement une propriété `loyaltyPoints` à l'objet.

Affiche le nom complet et les points de fidélité.

Étape 15 – Console : logs, warnings, errors

Objectif : Utiliser différents niveaux de logs pour le débogage.

Consignes

Modifie tes fonctions pour utiliser :

- `console.log()` pour les informations générales (ajout au panier, calcul de total)
- `console.warn()` pour les avertissements (stock faible, quantité élevée demandée)
- `console.error()` pour les erreurs (produit introuvable, stock insuffisant)

Exemple :

```
function addToCart(product, quantity) {
  if (!canAddToCart(product, quantity)) {
    console.error("Impossible d'ajouter au panier : stock insuffisant");
    return;
```

```
    }
    if (product.stock < 5) {
      console.warn("Attention : stock faible pour ce produit");
    }
    cart.push({ product, quantity });
    console.log(`${product.name} ajouté au panier (x${quantity})`);
  }
```

Étape 16 – Asynchrone : simulation de chargement

Objectif : Simuler le chargement des produits depuis une API.

Consignes

1. Crée une fonction `wait(ms)` qui retourne une promesse résolue après `ms` millisecondes :

```
function wait(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}
```

2. Crée une fonction `async loadProducts()` qui :
   - Affiche `"Chargement des produits..."`
   - Attend 2 secondes avec `await wait(2000)`
   - Affiche `"Produits chargés !"`
   - Retourne le tableau `products`
3. Utilise cette fonction pour charger les produits au démarrage :

```
async function init() {
  const loadedProducts = await loadProducts();
  console.log(`${loadedProducts.length} produits disponibles`);
  displayCatalog();
}

init();
```

Étape 17 – Code lisible et commentaires

Objectif : Nettoyer et documenter le code.

Consignes

1. **Indentation et nommage :**
   - Vérifie que tout ton code est bien indenté (2 ou 4 espaces)

    - Vérifie que toutes tes variables et fonctions ont des noms clairs et explicites

2. **Commentaires pertinents :**
    - Ajoute un commentaire en haut du fichier expliquant le but du projet
    - Ajoute des commentaires pour les parties complexes (calculs, logique métier)
    - Évite les commentaires évidents qui répètent le code
3. **Fonctions courtes :**
    - Si une fonction fait plus de 15-20 lignes, essaie de la découper en fonctions plus petites

---

Bonus – Étape 18 : Fonctionnalités supplémentaires

Si tu as terminé toutes les étapes, ajoute les fonctionnalités suivantes :

1. **Système de promotion :**
    - Crée une fonction `applyPromoCode(code)` qui applique une réduction si le code est valide
2. **Historique des commandes :**
    - Crée une fonction `checkout()` qui vide le panier et ajoute la commande à l'historique du client
3. **Tri des produits :**
    - Crée une fonction `sortProductsByPrice(ascending = true)` qui trie les produits par prix
4. **Recherche :**
    - Crée une fonction `searchProducts(query)` qui recherche dans les noms de produits
5. **Stock management :**
    - Crée une fonction `restockProduct(id, quantity)` qui ajoute du stock à un produit

Structure finale attendue

À la fin du projet, ton fichier `shop.js` devrait contenir :

- Variables globales (shopName, products, cart, customer)
- Fonctions utilitaires (calculs, vérifications)
- Fonctions de gestion du panier (add, remove, total)
- Fonctions de recherche et filtrage
- Fonction asynchrone de chargement
- Fonction d'initialisation
- Code propre, commenté et organisé

Bon courage !